ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΗΛΕΚΤΡΟΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ

ΕΡΓΑΣΤΗΡΙΟ ΜΙΚΡΟΫΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

ΣΥΣΤΗΜΑΤΑ ΜΙΚΡΟΫΠΟΛΟΓΙΣΤΩΝ

Αναφορά για την 1η Ομάδα Ασκήσεων

Μέλη Ομάδας

Κωνσταντόπουλος Γεώργιος (03122104)

Πουλόπουλος Ηλίας (03122160)

ΖΩΓΡΑΦΟΥ ΑΠΡΙΛΙΟΣ 2025

Ασκήσεις Προσομοίωσης

1η Άσκηση (Disassembly)

Βάσει των πινάκων στο *Παράρτημα 2* των σημειώσεων για το Microlab Simulator εξάγουμε τα παρακάτω:

Γλώσσα Μηχανής	Εντολές Assembly	Όρισμα εντολής
0E	MVI C, **Byte**	**08H**
3A	LDA **adr**	**2000H** (dip switches input)
17	RAL	-
DA	JC **adr**	**080DH** (END)
0D	DCR C	-
C2	JNZ **adr**	**0805H** (LOOPA)
79	MOV A,C	-
2F	CMA	-
32	STA **adr**	**3000H** (output LEDs)
CF	RST 1	-

Οι διευθύνσεις, όπως γνωρίζουμε από την θεωρία, σε γλώσσα μηχανής χωρίζονται σε δύο μέρη: address_low και address_high. Στη γλώσσα μηχανής εμφανίζεται πρώτα η low και μετά η high, για αυτό και αντιστρέφουμε τη σειρά για να πάρουμε το ζητούμενο address για να βάλουμε ως όρισμα σε μία εντολή (π.χ. **3A** 00 20 είναι **LDA** 2000).

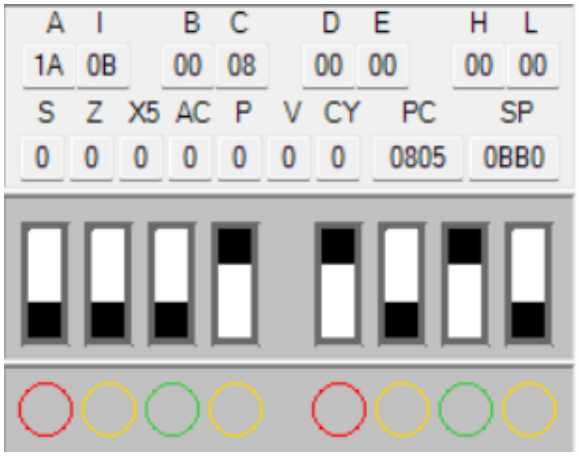
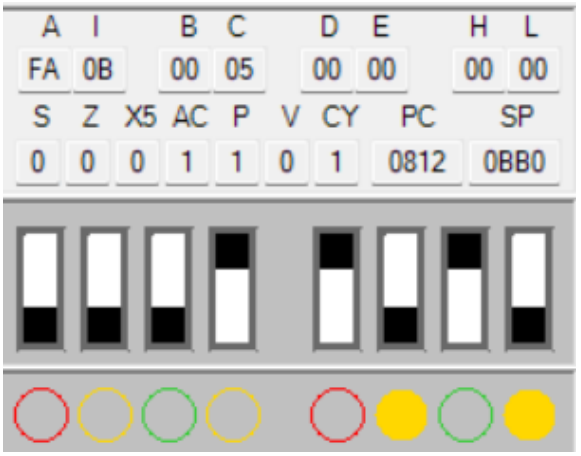
Γράφοντας ακριβώς τις εντολές όπως προέκυψαν από τη μετάφραση αυτή πήραμε το "**Αρχικό Πρόγραμμα",** το οποίο ήθελε διορθώσεις για να μεταφραστεί. Χρειάζεται αρχικά στο τέλος μία εντολή **END** και στα **Jump** οι διευθύνσεις θα πρέπει να αντικατασταθούν από tags. Για αυτό και το πρόγραμμα το μεταφράζουμε και έπειτα πατάμε το κουμπί **"Πρόγραμμα->Μνήμη και εντολές"** για να δούμε σε ποιές διευθύνσεις πρέπει να μεταπηδούν οι jump. Έτσι προκύπτει το πρόγραμμα "**Μνήμη και Εντολές".**

Χρειάστηκε λοιπόν η εισαγωγή 2 tag: **LOOPA** *(όχι LOOP γιατί χρησιμοποιείται ήδη από το Simulator)* και **END.** Έτσι λοιπόν, προέκυψε και το **"Τελικό Πρόγραμμα".**
Τα παραπάνω προγράμματα φαίνονται ευκρινώς παρακάτω:

Αρχικό Πρόγραμμα	Μνήμη και Εντολές	Τελικό Πρόγραμμα *(ex1.1.8085)*
MVI C,08H LDA 2000H RAL JC 080DH DCR C JNZ 0805H MOV A,C CMA STA 3000H RST 1	0800 0E MVI C,08H 0801 08 0802 3A LDA 2000H 0803 00 0804 20 LOOPA: 0805 17 RAL 0806 DA JC END 0807 0D 0808 08 0809 0D DCR C 080A C2 JNZ LOOPA 080B 05 080C 08 END: 080D 79 MOV A,C 080E 2F CMA 080F 32 STA 3000H 0810 00 0811 30 0812 CF RST 1	MVI C,08H ;C=8 (decimal) -> Loop Counter LDA 2000H ;Dip Switches Input LOOPA: RAL ;Left Rotate JC END ;If CY=1 end (When the first 1 is found) DCR C JNZ LOOPA END: MOV A,C ;A=(8-Loops_Done) CMA ;Complement of A as LEDs turn on on 0s STA 3000H ;LEDs Output RST 1 END

Ουσιαστικά το παραπάνω πρόγραμμα θέτει τον συσσωρευτή C ως ένα είδος loop counter θέτοντάς το ίσο με το δεκαδικό 8 (όσα και τα bits του input). Έπειτα διαβάζει την είσοδο από τα dip switches και κάνει σε αυτή συνεχής στροφές αριστερά έως ότου το Carry της στροφής αυτή γίνει ‘1’ (δηλαδή σταματάει μόλις βρει τον πρώτο άσσο στο input. Κάθε φορά που γίνεται αυτό, το C μειώνεται. Στην έξοδο εμφανίζεται το υπόλοιπο του loop counter, δηλαδή **ο αριθμός του bit (σε δυαδική μορφή) όπου βρέθηκε ο πρώτος άσσος** *(η αρίθμηση ξεκινάει από δεξιά προς τα αριστερά με το LSB να είναι το #1 bit).*

π.χ. Για input=(000**1** 1010)$_{BIN}$ = (1A)$_{HEX}$ παίρνουμε output= (0000 0101)$_{BIN}$ = (**5**)$_{DEC}$
Πράγματι, στο 5ο bit από δεξιά εμφανίζεται ο πρώτος άσσος. Αυτό φαίνεται ευκρινώς και στο παρακάτω screenshot

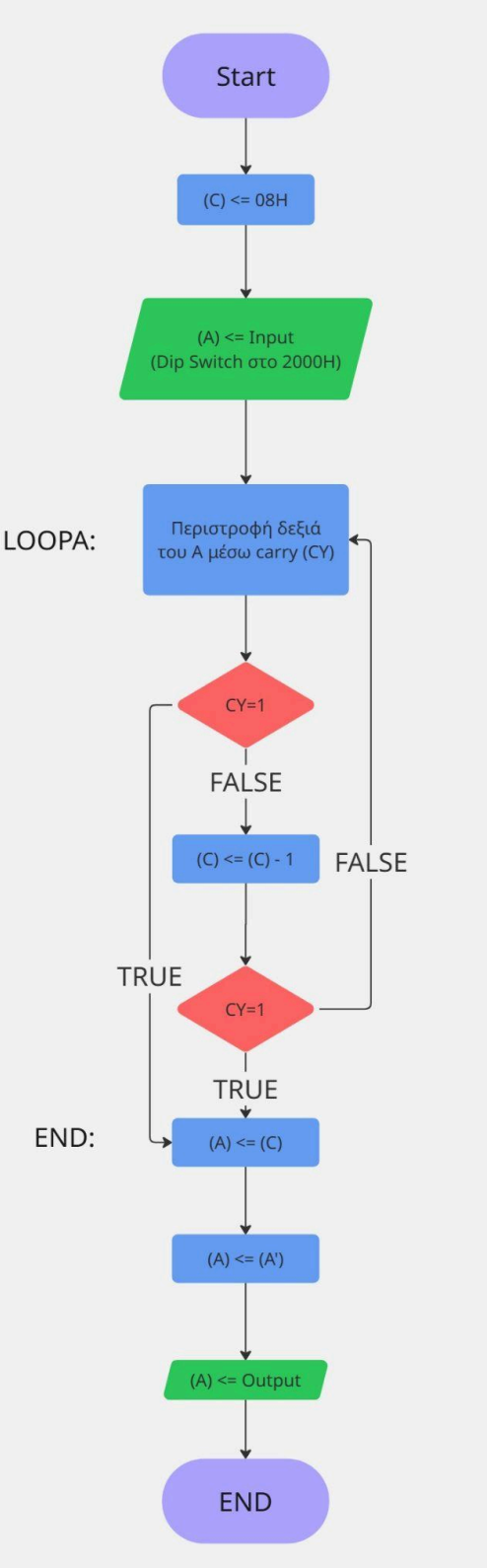
Input	Output
	

Το πρόγραμμα λειτουργεί μόνο για ένα προκαθορισμένο προ της εκτέλεσής του input και για αυτό μπορεί να χρειαστεί να το κάνουμε να τρέχει επ' άπειρον έτσι ώστε να δίνει output ακόμα και όταν κατά τη διάρκεια εκτέλεσής του αλλάζει το input από τα dip switches. Αυτό επιτυγχάνεται με την εισαγωγή μιας ετικέτας **START** στην αρχή του προγράμματος και η αντικατάσταση της **RST 1** από μία **JMP START.** Το εν λόγω πρόγραμμα φαίνεται στην επόμενη σελίδα.

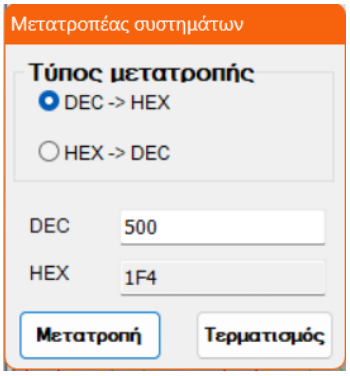
Δεξιά αυτού του προγράμματος απεικονίζεται το διάγραμμα ροής του αρχικού προγράμματος έτσι όπως το σχεδιάσαμε χρήσει του online εργαλείου Miro.

Πρόγραμμα που τρέχει επ' άπειρον ***(ex1.1-nonstop.8085)***	Διάγραμμα ροής

```
START:
MVI C,08H ;C=8 (decimal) -> Loop Counter
LDA 2000H ;Dip Switches Input

LOOPA:
RAL ;Left Rotate
JC END ;If CY=1 end (When the first 1 is found)
DCR C
JNZ LOOPA

END:
MOV A,C ;A=(8-Loops_Done)
CMA ;Complement of A as LEDs turn on on 0s
STA 3000H ;LEDs Output
JMP START
END
```



2η Άσκηση (LEDs left/right)



Αρχικά για την DELB, από τη στιγμή που θέλουμε καθυστέρηση μισού δευτερολέπτου θα πρέπει να πολλαπλασιάσουμε το 1ms με το περιεχόμενο του ζεύγους BC, που στη προκειμένη περίπτωση θα θέλαμε να είναι ίσο με 500.

500 x 1ms = ½ sec

Χρήσει του μετατροπέα βρίσκουμε πως στην προκειμένη περίπτωση θα φορτώσουμε το 01F4H.

Η γενική ιδέα του προγράμματος φαίνεται στα σχόλια στο παρακάτω πρόγραμμα ***(ex1.2.8085)*** και η ορθή λειτουργία του μπορεί να διαπιστωθεί και από το παρακάτω βίντεο: ex1.2 Simulation

Τα σχόλια στην αναφορά είναι στα ελληνικά, αλλά στο αρχείο του προγράμματος είναι στα αγγλικά

```
IN 10H
MVI D,FEH ;(1111 1110)->LSB LED ON
MOV A,D
STA 3000H ;LSB LED ON
LXI B,01F4H ;500 decimal -> 1ms*500=1/2 sec for DELB

START:
LDA 2000H ;Dip Switch Input
RRC ;Παίρνουμε το LSB
JNC PAUSE ;If CY=LSB=0 then pause, else...
RLC ;2 αριστερά rotations ώστε να πάρουμε το MSB, εφόσον ήδη έγινε δεξί rotation
RLC
JC RIGHT ;If CY=MSB=1 δεξιόστροφη κίνηση του LED που είναι ανοιχτό

;Διαφορετικά κάνει αριστερόστροφη κίνηση του LED που είναι ανοιχτό
MOV E,A ;Αποθήκευση του input στο E
MOV A,D ;Το D είναι το bit pattern που έχει 1 LED ON (7 άσσοι και 1 μηδενικό)
RLC ;Με αριστερό rotation το '0' μετακινείται αριστερά
STA 3000H ;LED Output of the bit pattern
MOV D,A ;Αποθηκεύει το νέο μοτίβο bits στον D
CALL DELB
MOV A,E ;Επαναφέρει στον A το αρχικό input
JMP START ;Επανελέγχει αν άλλαξε το input από τα dip switches και κάνει το ίδιο
```

```
;Όμοια για την δεξιόστροφη κίνηση
RIGHT:
MOV E,A
MOV A,D
RRC
STA 3000H
CALL DELB
MOV D,A
MOV A,E
JMP START

PAUSE:
CALL DELB
JMP START
END
```

3η Άσκηση (BIN->BCD)

Παίρνουμε από τις διαφάνειες (καθώς το βιβλίο ακόμα και μέχρι τις αρχές του Απριλίου δεν το είχαμε παραλάβει γιατί ο Εύδοξος δεν ανοίγει ποτέ στην ώρα του 🙂) το παρακάτω πρόγραμμα για τη μετατροπή ενός δυαδικού αριθμού σε δεκαδική μορφή δύο ψηφίων (BCD) με περιορισμό να είναι μικρότερος του 99.

```
        LDA 0042H
        CPI 64H       ; Είναι μεγαλύτερος του 99? (A-99)
        JNC END       ; Αν ναι τέλος (A>99)
        MVI B, FFH     ; Αλλιώς A ≤ 99
DECA:
        INR B
        SUI 0AH       ; Αλλεπάλληλες αφαιρέσεις του 10
        JNC DECA      ; Αν είναι θετικός συνέχισε
        ADI 0AH       ; Διόρθωση του αρνητικού υπολοίπου
        STA 0040H     ; Αποθήκευση μονάδων
        MOV A,B
        STA 0041H     ; Αποθήκευση δεκάδων
END:
        HLT
```

Στον κώδικα BCD με 8 bits μπορούμε να απεικονίσουμε όλους τους αριθμούς από το 00 έως το 99. Το binary input μετατρέπεται σε διψήφιο δεκαδικό και το κάθε ψηφίο του μετατρέπεται σε δυαδικός. Έτσι τα 4 πρώτα bit ενός BCD αναπαριστά το πρώτο ψηφίο του δεκαδικού αριθμού (δεκάδες) και τα 4 επόμενα αναπαριστά το δεύτερο ψηφίο (μονάδες).

π.χ. **$0001\ 1110_{BIN} = 30_{DEC}$ => $3_{DEC}=0011_{BIN}$, $0_{DEC}=0000_{BIN}$ => BCD: 0011 0000**

Με αναφορά το δοσμένο πρόγραμμα, παράχθηκε ο παρακάτω κώδικας ***(ex1.3.8085)***

```
START:
LDA 2000H ;Input από Dip Switches
LXI B,03E8H ;1000 decimal -> 1 sec delay για DELB.
CPI 00H ;Ελέγχει αν input=0
JZ START ;Αν είναι τότε ξαναπαίρνει το input έως ότου γίνει !=0

CPI C8H ;Ελέγχει αν input>199 πρώτα
JNC MSBLED ;Αν είναι τότε το MSB LED αναβοσβήνει

CPI 64H ;Ελέγχει αν input>99
JNC LSBLED ;Αν είναι τότε το MSB LED αναβοσβήνει

;Αν input<99
MVI B,FFH ;B=(1111 1111) -> All LEDs OFF

DECA:
INR B ;B++. Ο B κρατάει τον αριθμό των αφαιρέσεων που μας δίνει τις δεκάδες
;B=0000 DDDD (D: Πρώτο ψηφίο του δεκαδικού διψήφιου αριθμού σε δυαδική μορφή)
SUI 0AH ;Συνεχείς αφαιρέσεις του 10 από τις οποίες προκύπτουν οι μονάδες
JNC DECA  ;If CY=0 αφαίρεσε ξανά 10
ADI 0AH ;If CY=1 πρόσθεσε 10 (Διόρθωση θετικού κρατούμενου)
MOV C,A ;Αποθήκευση των μονάδων (U: Δεύτερο ψηφίο). C=0000 UUUU
MOV A,B ;A=0000 DDDD
RLC ;4 αριστερές περιστροφές για να πάρουμε τις δεκάδες στα 4 πρώτα bits
RLC ;A=DDDD 0000
RLC
RLC
ADD C ;Πρόσθεση των μονάδων (0000 UUUU) στον A. A=DDDD UUUU
CMA ;Αφού τα LED ανάβουν για '0' παίρνουμε το συμπλήρωμα του A
STA 3000H ;LED Output
CALL DELB
JMP START ;Επανέλεγχος Input
```

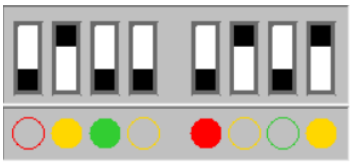
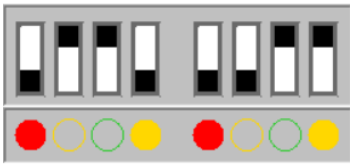
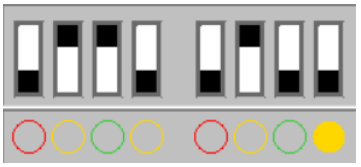
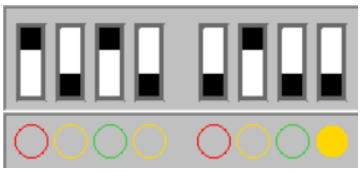
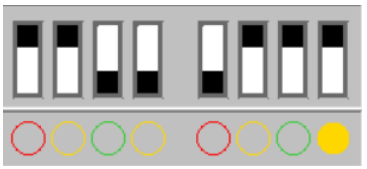
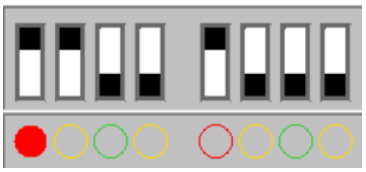
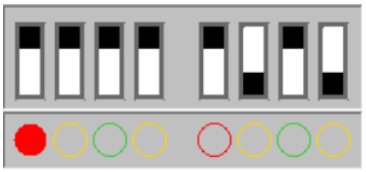
```
;MSB LED blink
MSBLED:
MVI A,7FH ;A=0111 1111 -> MSB LED ON
STA 3000H ;LED Output
CALL DELB
MVI A,FFH ;A=1111 1111 -> All LEDs OFF
STA 3000H ;LED Output με το MSB OFF
CALL DELB
JMP START ;Επανέλεγχος του input και αν δεν άλλαξε ξανά το MSB θα κάνει blink

;LSB LED blink
LSBLED:
MVI A,FEH ;A=1111 1110 -> LSB LED ON
STA 3000H ;LED Output
CALL DELB
MVI A,FFH ;A=1111 1111 -> All LEDs OFF
STA 3000H ;LED Output με το LSB OFF
CALL DELB
JMP START ;Επανέλεγχος του input και αν δεν άλλαξε ξανά το LSB θα κάνει blink

END
```

Τα σχόλια στην αναφορά είναι στα ελληνικά, αλλά στο αρχείο του προγράμματος είναι στα αγγλικά.

Η συνεχής λειτουργία μπορεί να επιβεβαιωθεί και από το παρακάτω βίντεο

[ex1.3 Simulation]

Παραδείγματα

Κατηγορία	DEC	BIN	BCD/LED	Output
<99	69	0100 0101	0110 1001	
=99	99	0110 0011	1001 1001	
>99	100	0100 0100	LSB	
>99	164	1010 0100	LSB	
=199	199	1100 0111	LSB	
>199	200	1100 1000	MSB	
>199	250	1111 1010	MSB	

Θεωρητικές Ασκήσεις

4η Άσκηση (Κόστος Κατασκευής)

Δεδομένα

1η τεχνολογία: I.C. σε σχετικά μεγάλη πλακέτα

- Αρχικό = 20.000
- Κόστος-ICs = 10
- Κόστος κατασκευής = 10
- **Κόστος=20.000+20t=K_1(t)**

2η τεχνολογία: FPGAs

- Αρχικό = 10.000
- Κόστος-ICs = 30
- Κόστος κατασκευής = 10
- **Κόστος=10.000+40t=K_2(t)**

3η τεχνολογία: SoC-1 σε μικρή πλακέτα

- Αρχικό = 100.000
- Κόστος-ICs = 2
- Κόστος κατασκευής = 2
- **Κόστος=100.000+4t=K_3(t)**

4η τεχνολογία: SoC-2 σε πολύ μικρή πλακέτα

- Αρχικό = 200.000
- Κόστος-ICs = 1
- Κόστος κατασκευής = 1
- **Κόστος=200.000+2t=K_4(t)**

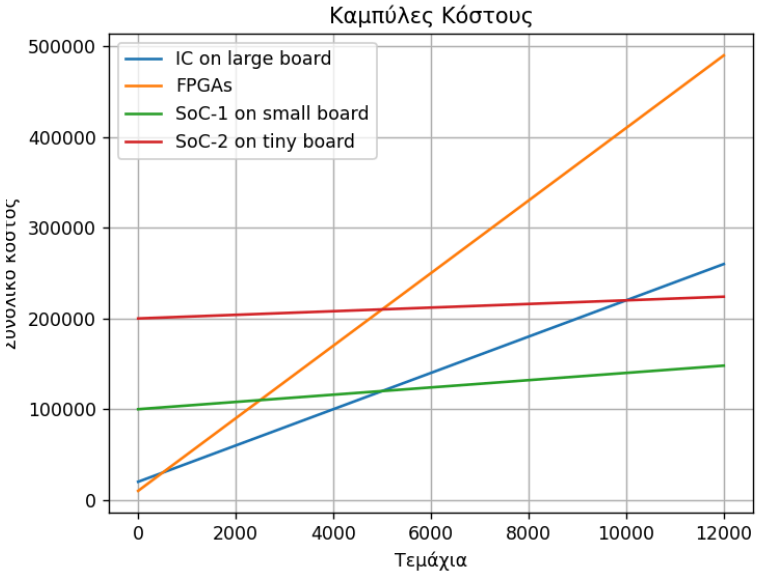
Για την εύρεση των γραφικών παραστάσεων κόστους και των καμπυλών κόστους ανά τεμάχιο βάσει της σχέσης:

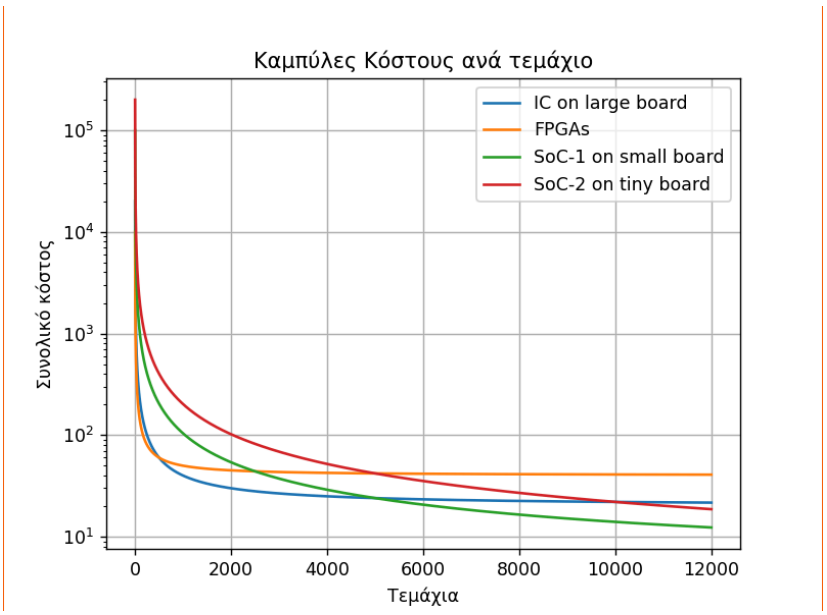
Κόστος= Αρχικό + (Κόστος-ICs + Κόστος-κατασκευής)×Πλήθος τεμαχίων

γράφτηκε ο παρακάτω κώδικας σε **Python**:

```
import matplotlib.pyplot as plt
import numpy as np

t=np.linspace(0,11999,12000) #t: Πλήθος τεμαχίων
initial_cost=[20000, 10000, 100000, 200000] #Αρχικό κόστος κατασκευής
ics_cost=[10, 30, 2, 1] #Κόστος των ICs
construction_cost=[10, 10, 2, 1] #Κόστος κατασκευής

cost=[]
for i in range(len(initial_cost)):
    c=initial_cost[i] + (ics_cost[i] + construction_cost[i]) * t
    cost.append(c)

labels = ["IC on large board", "FPGAs", "SoC-1 on small board", "SoC-2
on tiny board"]
for i in range(4):
    plt.plot(t, cost[i], label=labels[i])
plt.xlabel('Τεμάχια')
plt.ylabel('Συνολικό κόστος')
plt.title('Καμπύλες Κόστους')
plt.grid()
plt.legend()
plt.savefig("ex4.1-1.jpg")
plt.show()

cost_perunit=[]
for i in range (len(initial_cost)):
    cpu=(initial_cost[i]/t) + ics_cost[i] + construction_cost[i]
    cost_perunit.append(cpu)

for i in range(4):
    plt.plot(t, cost_perunit[i], label=labels[i])
plt.xlabel('Τεμάχια')
plt.ylabel('Συνολικό κόστος')
plt.yscale('log')
plt.title('Καμπύλες Κόστους ανά τεμάχιο')
plt.grid()
plt.legend()
plt.savefig("ex4.1-2.jpg")
```

Παρακάτω απεικονίζονται οι καμπύλες κόστους και οι καμπύλες κόστους ανά τεμάχιο (σε λογαριθμική κλίμακα):





Παρατηρώντας τις γραφικές παραστάσεις κόστους εξάγουμε τις παρακάτω ανισώσεις:

- **$K_2(t) < K_1(t) \Rightarrow t <= 50$** τεμάχια (FPGAs συμφερότερα)
- **$K_1(t) < K_3(t) \Rightarrow t <= 5.000$** τεμάχια (I.Cs συμφερότερα)
- **$K_3(t) < K_4(t) \Rightarrow t <= 50.000$** τεμάχια (SoC-1 συμφερότερο)
- **Για t>50.000** (SoC-2 συμφερότερο)

Για την περίπτωση των FPGAs η τιμή ανα τεμάχιο IC που θα μπορούσε να εξαφανίσει την επιλογή της 1ης τεχνολογίας μετά από κατάλληλη επεξεργασία των εξισώσεων είναι **10**.